Fondamentaux du langage Python

- Variables / Types
- Entrées / Sorties
- Chaînes de caractères
- Structures conditionnelles
- Boucles

Coordinatrice : Cherifa BEN KHELIL
cherifa.ben-khelil@efrei.fr
Support du cours: Asma Gabis

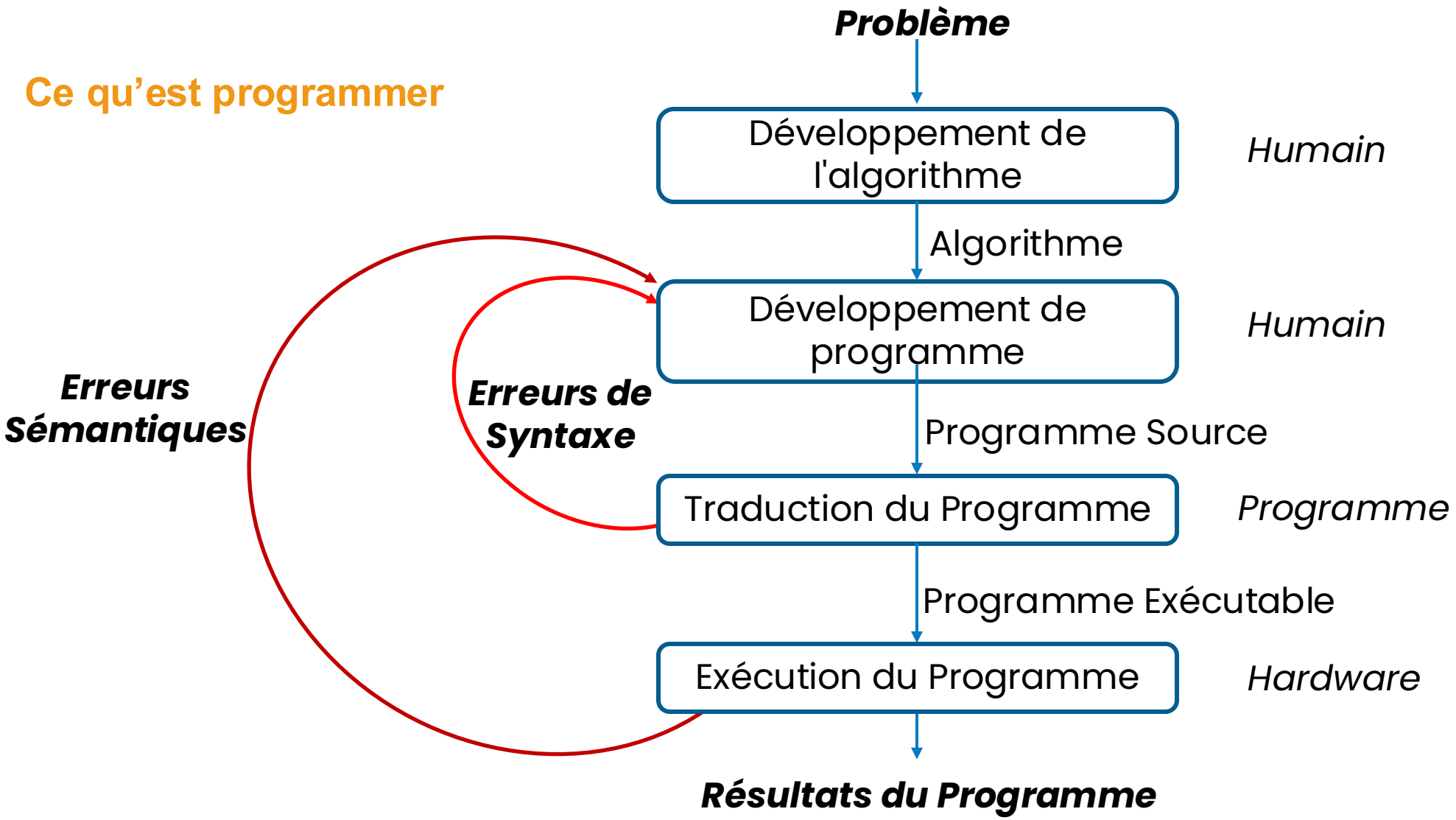
01

Le langage Python

01

La programmation

Ce qu'est programmer



La programmation

Erreurs syntaxiques Vs Erreurs sémantiques

- **Erreur de Syntaxe** : violation d'une règle du langage de programmation (grammaire).
  - *Exemple:* "Moi parle Anglais bien."
  - Utiliser des mots clef valides mais au mauvais endroit
  - Détecté par un compilateur

- **Erreur de Sémantique** : erreur logique :
  - *Exemple:* "Cette phrase est écrite en Italien"
  - Un programme syntaxiquement correct mais ne fournissant pas le résultat attendu
  - L'utilisateur observe la sortie de l'exécution d'un programme

Le langage Python

Définition

- Développé par Guido van Rossum au début des années 1990.
- A télécharger http://www.python.org
- Langage interprété : fonctionne avec un évaluateur d'expressions du langage
- Langage orienté objet : tout est un objet
- Typage dynamique : les variables n'ont pas un type prédéfini

Le langage Python

Pourquoi Python ?

- Langage simple à apprendre.
- Langage populaire
- Langage utilisé en production
- Multiplateformes
- Très large champ d’application

Le langage Python

Modes de programmation en python

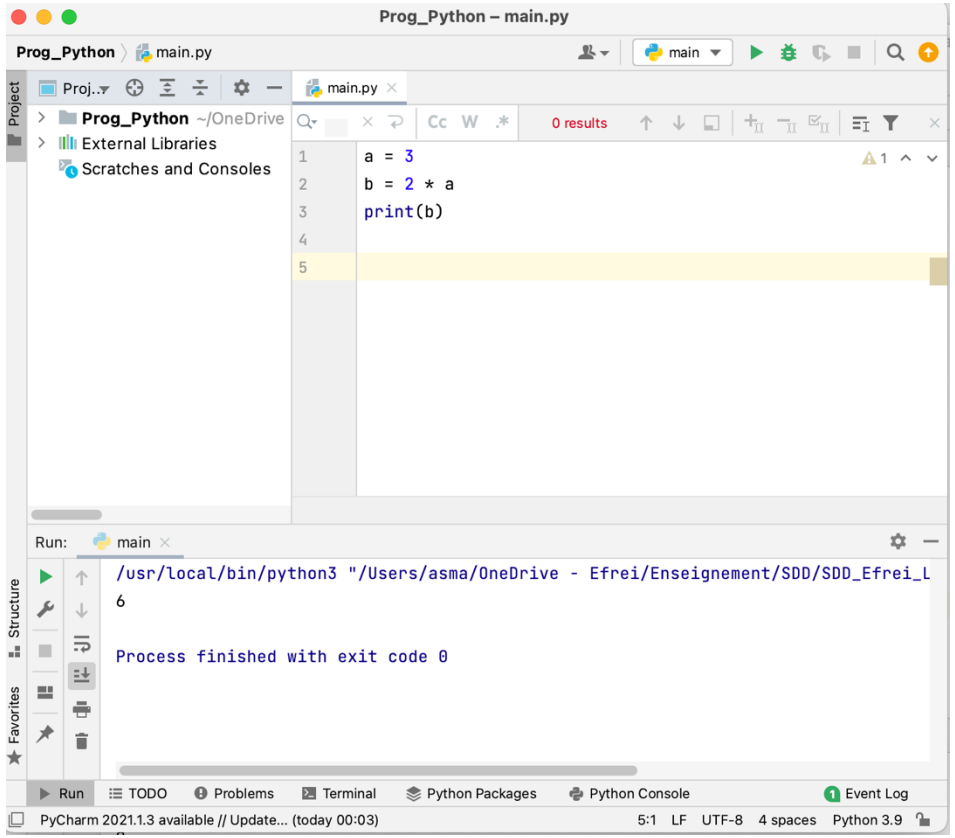
Mode interpréteur (console) :

exécution des instructions en temps réel MAIS pas possible d'exécuter plusieurs blocs d'instructions.

```
>>> a = 3
>>> b = 2 * a
>>> b
6
```

Marque de programmation en mode console

Mode environnement de développement (IDE) :



Modes de programmation en python

Dans ce cours, seul le mode IDE sera abordé.

L'IDE à utiliser est: **PyCharm**

02

Les bases de la programmation Python

02

Les bases de la programmation Python

Structure d'un programme Python

Indentation : Fait référence à l'espace mis en début d'une ligne de code

- Réalisable grâce à la touche « tabulation » du clavier
- Fait partie de la syntaxe du langage Python , elle est donc obligatoire.
- L'indentation sur Python sert à délimiter les blocs d'instructions

Exemple

```
a = 3
b = 5
c = 10
if a > b :
    if a > c :
        print(a)
    else:
        print (b)
```

Les bases de la programmation Python

Structure d'un programme Python

Commentaires : Parties du code non prises en considération à l'exécution d'un programme

- Utilisés pour rendre le code plus lisible
- Utilisés pour expliquer une portion de code
- Utilisés pour ignorer une partie du code à l'exécution

Comment ça marche ?

- # : utilisé pour commenter une ligne
- ''' code ''' : Commenter plusieurs lignes en utilisant 3 apostrophes au début du code à commenter et 3 apostrophes à la fin.
- « « « code » » » : Commenter plusieurs lignes en utilisant 3 double apostrophes (guillemets) au début du code à commenter et 3 double apostrophes à la fin.

Exemple

```
#Ceci est un commentaire
a = b = 3
c = 10
'''
if a > b :
    if a > c :
        print(a)
'''
```

Les bases de la programmation Python

Variables / Types

Variable : Conteneur (boîte) pour y stocker une valeur :

- Nombre : pour faire des opérations mathématiques (additions, calcul TVA, ..)
- Mot (chaîne de caractères) : pour former des phrases.

Type : Détermine la nature de la variable :

- Une variable d'un type donné peut prendre plusieurs valeurs mais pas plusieurs types
- Plusieurs types de variables sont présent : Type primitifs

Les bases de la programmation Python

Les types primitifs

Primitive	Signification	Taille (en octets)	Plage de valeurs acceptée
int	Entier	4	-2 147 483 648 à 2 147 483 647
float	flottant (réel)	8	10^{-308} à 10^{308}
str	Chaîne de caractères		
bool	booléen	1	False ou True (toute valeur différente de 0 est considérée comme True)

Syntaxe exacte du type

Informations sur le type

Manipulation des variables

Déclaration d'une variable

Typage dynamique : pas de déclaration explicite de la variable.

Python **devine** le type de la variable « Nom_variable » à partir du type de la valeur affectée.

Nom_variable = valeur

Nom symbolique donnée à une variable: choisir des noms significatifs.

Initialisation de la variable: première valeur attribuée à une variable.

Les bases de la programmation Python

Manipulation des variables

Exemples

```
num_etudiant = 5 # num_etudiant sera considérée de type int

PI = 3.14 # PI sera considérée comme une variable de type float

prenom = "Maria" # prenom est une variable de str ici

est_valide = True # Cette variable est de type bool
```

Les bases de la programmation Python

Le type str

- Une chaîne de caractères est représentée en Python par:
  - Doubles quotes (ou guillemets) « » : `cours = "python"`
  - Simples quotes (ou apostrophes) ‘ ‘ : `cours = 'python'`

- Si l’apostrophe fait partie de la chaîne de caractères:
  - Utiliser des guillemets « »: `mot = "n’est"`

- Si la chaîne de caractères contient des guillemets:
  - Utiliser des triples guillemets « « « » » »: `mot = """a'b"c"""`

Les bases de la programmation Python

Nommage des variables

Il est autorisé de:

- Utiliser les lettres minuscules (a – z)
- Utiliser les lettres majuscules (A – Z)
- Utiliser les chiffres (0 – 9)
- Le caractère de soulignement (_)

Il n'est pas autorisé de:

- Commencer le nom d'une variable par un chiffre
- Utiliser des espaces dans les noms des variables
- Utiliser des accents
- Utiliser les mots réservés du langage: `print`, `def`, `and`, `or`, `for`, …

Python est <u>sensible à la casse</u>.

Exemple:

Les variables nommées `NOTE`, `Note`, `note` sont différentes.

Les bases de la programmation Python

Les opérateurs

L'affectation

Symbolisée par le caractère **=** . Elle permet l'attribution d'une valeur à une variable.

Exemple: `x = 5`

L'affectation multiple

Python donne la possibilité d'affecter des valeurs à plusieurs variables sur une seule ligne.

Exemple:

```
x, y, z = 5, 3.5, "test"
```

Equivalent à

```
x = 5
y = 3.5
z = "test"
```

```
a = b = 3
```

Equivalent à

```
b = 3
a = b
```

Les opérateurs

L'affectation : Syntaxe Vs erreurs à éviter

Nom_variable = valeur

Une valeur: 2, 12.8, «True », …

Une expression:
A + b, x * y + z, …

MAIS

5 = y

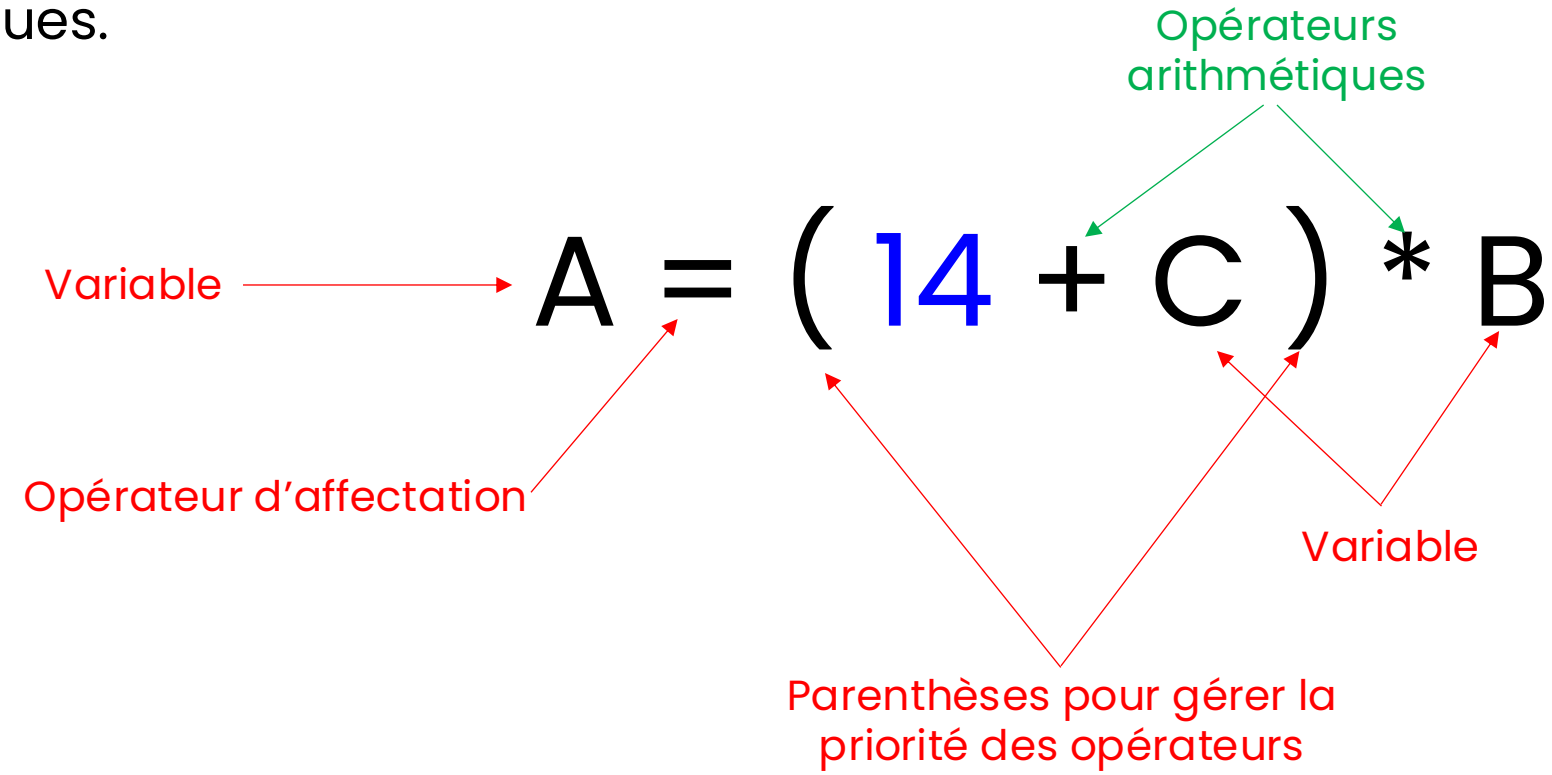
a + b = y

a + 1 = b

Les opérateurs

Expression arithmétique

Formée par une combinaison d'objets numériques (int ou float) et des opérateurs arithmétiques.



Les bases de la programmation Python

Les opérateurs

Les opérateurs arithmétiques

Opérateur	Dénomination	Effet	Exemple	Résultat (avec x valant 7)
+	opérateur d'addition	Ajoute deux valeurs	x+3	10
-	opérateur de soustraction	Soustrait deux valeurs	x-3	4
*	opérateur de multiplication	Multiplie deux valeurs	x*3	21
/	opérateur de division	Divise deux valeurs	x/3	2.3333333
%	Opérateur du modulo	Reste de la division	x%3	1
//	Division entière	Quotient d'une division	x//3	2
**	Opérateur de puissance	Calcul de la puissance	x**2	49

Les opérateurs

Les opérateurs arithmétiques composés

Opérateur	Opération composée	Opération normale
+=	X += Y	X = X + Y
-=	X -= Y	X = X - Y
***=**	X *= Y	X = X * Y
/=	X /= Y	X = X / Y
//=	X //= Y	X = X // Y
%=	X %= Y	X = X % Y
****=**	X **= Y	X = X ** Y

Les opérateurs

Expression de comparaison

Donne un résultat booléen : True ou False.

Opérateur de comparaison

Variable booléenne

```
A = 3 >= (4 * B)
```

Expressions arithmétiques

Les opérateurs

Opérateurs de comparaison

Opérateur	Dénomination	Effet	Exemple	Résultat
== **A ne pas confondre avec le signe d'affectation (=)!!**	opérateur d'égalité	Compare deux valeurs et vérifie leur égalité	x==3	Retourne True si x est égal à 3, sinon False
<	opérateur d'infériorité stricte	Vérifie qu'une variable est strictement inférieure à une valeur	x<3	Retourne True si x est inférieur à 3, sinon False
<=	opérateur d'infériorité	Vérifie qu'une variable est inférieure ou égale à une valeur	x<=3	Retourne True si x est inférieur ou égal à 3, sinon False
>	opérateur de supériorité stricte	Vérifie qu'une variable est strictement supérieure à une valeur	x>3	Retourne True si x est supérieur à 3, sinon False
>=	opérateur de supériorité	Vérifie qu'une variable est supérieure ou égale à une valeur	x>=3	Retourne True si x est supérieur ou égal à 3, sinon False
!=	opérateur de différence	Vérifie qu'une variable est différente d'une valeur	x!=3	Retourne True si x est différent de 3, sinon False

Les opérateurs

Expression logique

Donne un résultat booléen : True ou False.

Opérateur logique

Variable booléenne

$$A = (B > 3)\ \mathbf{and}\ (4 == B)$$

Expressions de comparaison

Les bases de la programmation Python

Les opérateurs

Opérateurs logiques

Opérateur	Dénomination	Effet	Syntaxe
or	OU logique	Vérifie qu'il y a au moins une condition réalisable	((condition1) **or** (condition2))
and	ET logique	Vérifie que toutes les conditions sont réalisées	((condition1) **and** (condition2))
not	NON logique	Inverse l'état d'une variable booléenne (retourne la valeur True si la variable vaut False, False si elle vaut True)	(**not** condition)

Les opérateurs

Tables de vérité

and		
A	**B**	**A and B**
False	False	False
False	True	False
True	False	False
True	True	True

or		
A	**B**	**A or B**
False	False	False
False	True	True
True	False	True
True	True	True

not	
A	**not A**
False	True
True	False

Les opérateurs

Opérateurs d'appartenance

Opérateur	Dénomination	Effet	Syntaxe
in	Test d'appartenance	True si la valeur / variable se trouve dans la séquence	5 in x
not in	Test de non-appartenance	True si la valeur / variable ne se trouve pas dans la séquence	5 not in x

Les opérateurs

Opérateurs d'identité

Opérateur	Dénomination	Effet	Syntaxe
is	Test d'égalité de types non primitifs	True si les opérandes sont identiques (se référer au même Type)	x is True
is not	Test de non-égalité de types non primitifs	True si les opérandes ne sont pas identiques (pas de référence pour le même Type)	x is not True

La concaténation

- Mettre bout à bout deux chaînes de caractères
- L'opérateur + reliant deux chaînes de caractères fait effet de concaténation

Expression arithmétique + Expression arithmétique

Opérateur d'addition

Exemple

```
a = 3
b = a + 3
```

b = 6

Chaîne de caractères + Chaîne de caractères

Opérateur de concaténation

Exemple

```
mot1 = "cours"
mot2 = "python"
mot = mot1 + mot2
```

mot = « courspython »

La répétition

- L'opérateur * appliqué à une chaînes de caractères permet de répéter cette chaîne de caractères un nombre de fois

Expression arithmétique * Expression arithmétique

Opérateur de multiplication

Exemple

```
a = 3
b = a * 3
```

b = 9

Nombre * Chaîne de caractères

Opérateur de répétition

Exemple

```
mot1 = "cours"
mot = 3 * mot1
```

mot = « courscourscours »

Casting

- Forcer la conversion d'une variable d'un type donné vers un autre type.
- Réalisable en utilisant les méthodes:
  - `int()`
  - `float()`
  - `str()`

Exemples

```
y = int(2.8) # y sera égal à 2
z = int("3") # z sera égal à 3
a = float("3") # a sera égal à 3.0
w = float("4.2") # w sera égal à 4.2
b = str(3.0)  # b sera égal à '3.0'
```

03

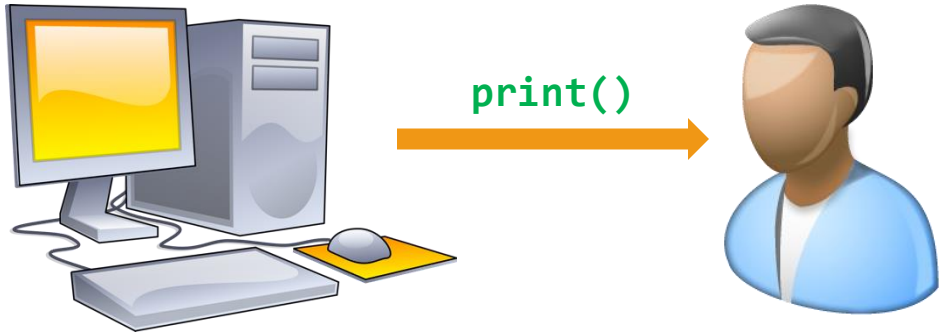
Les entrées / sorties en langage Python

03

Les entrées / sorties en Python

Affichage à l'écran

- `print()`: permet d'effectuer un affichage à l'écran. Il peut être:
  - Un message sous forme d'une chaîne de caractères entre quotes ou doubles quotes.
  - Un nombre ou une variable
  - Une expression arithmétique ou logique
  - Une hybridation de tous ces types



Les entrées / sorties en Python

Affichage à l'écran

Les différentes parties d'un print sont séparées par des **VIRGULES**

```
print("message à afficher", val1, val2, "autre message", val1 == val2)
```

Messages

Variables

Expression

Les entrées / sorties en Python

Affichage à l'écran

- Un retour à la ligne est appliqué automatiquement après un `print()`
- Pour empêcher un retour à la ligne automatique, ajouter le paramètre **end** au `print()` en précisant un caractère de fin de ligne différent du retour à la ligne.
- Le caractère « espace » doit être mis entre guillemets pour voir son effet appliqué à l'écran.

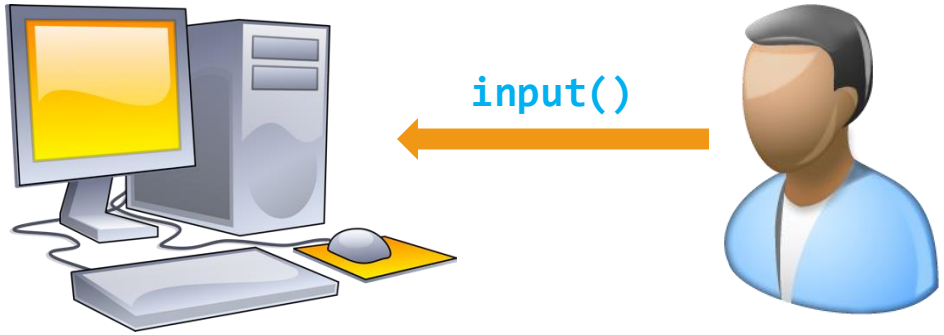
```
# permet d'afficher: Bonjour le monde
print ("Bonjour", end=" ") # Insertion d'un espace après Bonjour grâce au paramètre end
print ("le monde") # retour à la ligne automatique après affichage de "le monde"
```

```
# permet d'afficher: Bonjour;le monde
print ("Bonjour", end=';') # Insertion d'un point-virgule après Bonjour grâce au paramètre end
print ("le monde") # retour à la ligne automatique après affichage de "le monde"
```

Les entrées / sorties en Python

Saisie de données

- `input()` permet de demander à l'utilisateur de fournir des informations.
- **Chaque information donnée par l'utilisateur est stockée dans une variable**.
- À l'exécution du `input()`, la machine se met en attente jusqu'à ce que l'utilisateur saisisse une valeur.
- **L'information stockée à partir d'un `input()` est toujours de type str**.



Les entrées / sorties en Python

Saisie de données

Message indiquant à l'utilisateur l'information à saisir

Variable où sera stockée la valeur saisie par l'utilisateur

```
nom_var = input("Saisir une valeur: ")
```

L'information est saisie sur la MÊME LIGNE que le message

Presque équivalent

L'information est stockée sous forme d'un **str**

```
print("Saisir une valeur: ")
nom_var = input()
```

Après le message, un retour à la ligne est appliqué avant la saisir de l'information.

Les entrées / sorties en Python

Saisie de données

Comment faire pour saisir des données numériques ?

```
nom_var_int = int(input("Saisir une valeur: "))
```

Variable de type **int**

```
nom_var_float = float(input("Saisir une valeur: "))
```

Variable de type **float**

04

Les structures de contrôle

04

Les structures de contrôle

Structures conditionnelles

- if
- if - else
- if - elif - else

Boucles

- while
- for

Les structures de contrôle

Les structures conditionnelles

- **if : syntaxe**

Peut être:
- Condition simple: expression de comparaison
- Condition composée: expression logique

Mot clé : OBLIGATOIRE

```
if condition :
    action(s)
```

OBLIGATOIRE: fait partie de la syntaxe du langage

Indentation: OBLIGATOIRE. Toutes les actions à faire si la condition du if est True doivent être dans un bloc indenté

Une ou plusieurs action(s) exécutée(s) uniquement si condition est True

Les structures de contrôle

Les structures conditionnelles

- **if : exemples**

Ces deux instructions seront exécutées si la condition du « if » est True

```
if x <= 10 :
    y = x * x + 5
    z = (2 * y) / 3
```

```
if x <= 10 :
    y = x * x + 5
z = (2 * y) / 3
```

Seule cette instruction sera exécutée si la condition du « if » est True

Toutes les conditions du « if » doivent être à True pour exécuter cette instruction

```
if 0 <= age and age <= 11:
    kids = kids + 1
```

```
if mois==4 or mois==6 or mois==9 or mois==11:
    print("Le mois contient 30 jours")
```

Il suffit d'une seule condition du « if » à True pour exécuter cette instruction.

Les structures de contrôle

Les structures conditionnelles

- **if – else : syntaxe**

Peut être:
- Condition simple: expression de comparaison
- Condition composée: expression logique

Mots clé : OBLIGATOIRES

```
if (condition (s)):
    action(s)_if
else:
    action(s)_else
```

Indentation: OBLIGATOIRE.

OBLIGATOIRES: font partie de la syntaxe du langage

Les structures de contrôle

Les structures conditionnelles

- **if – else : exemples**

```
a = int(input("Saisir une valeur: "))
if (a > 0):
    print(a, "est strictement positif")
else:
    print(a, "est négatif ou nul")
```

```
a = int(input("Saisir une valeur: "))
if (a > 0):
    print(a, "est strictement positif")
else:
    if (a < 0):
        print(a, "est strictement négatif")
    else:
        print(a, "est nul")
```

JAMAIS de condition après else

Les structures de contrôle

Les structures conditionnelles

Syntaxe utilisée lorsqu'il y a plusieurs cas à tester: cas de « if » imbriqués

- **if – elif – else : syntaxe**

Mots clé : OBLIGATOIRES

```
if condition(s)1 :
    action(s)_if
elif condition(s)2 :
    action(s)_elif

    ....

elif condition(s)N :
    action(s)_elif :
else :
    action(s)_else
```

Indentation: OBLIGATOIRE.

OBLIGATOIRES: font partie de la syntaxe du langage

Les structures de contrôle

Les structures conditionnelles

- **if – elif – else : exemple**

```
a = int(input("Saisir une valeur: "))
if (a > 0):
    print(a, "est strictement positif")
else:
    if (a < 0):
        print(a, "est strictement négatif")
    else:
        print(a, "est nul")
```

```
a = int(input("Saisir une valeur: "))
if (a > 0):
    print(a, "est strictement positif")
elif (a < 0):
    print(a, "est strictement négatif")
else:
    print(a, "est nul")
```

Peut également s'écrire comme ça

Les structures de contrôle

Boucles

- **while: syntaxe**

```
initialisation
while condition(s) :
    action(s)
    incrémentation
```

Mot clé : OBLIGATOIRE

OBLIGATOIRES: fait partie de la syntaxe du langage

- ✓ Exécuter le corps de la boucle (action(s)) tant que la condition est vérifiée (le test vaut **True**)
- ✓ La condition est évaluée **avant** d'exécuter le corps de la boucle
- ✓ Le corps de la boucle peut contenir une ou plusieurs autres boucles: **Boucles imbriquées**

Les structures de contrôle

Boucles

- **while: exemples**

```
x = 2
while x < 5 :
    print(x, " est dans la boucle")
    x = x + 1
```

Exemple d'utilisation d'une boucle while

```
x = 2
while x < 5 :
    y = 1
    while y < 5 :
        print(x + y)
        y = y + 1
    x = x + 1
```

Exemple d'utilisation de boucles while imbriquées

Les structures de contrôle

Boucles

- **for: syntaxe**

Mot clé : OBLIGATOIRE
Permet à l'iterator de prendre chacune des valeurs de element

Mot clé : OBLIGATOIRE

```
for iterator in element :
    action(s)
```

OBLIGATOIRES: fait partie de la syntaxe du langage

« element » peut être :
- Un intervalle de valeurs
- Une chaîne de caractères
- Un tableau (liste en python)

Les structures de contrôle

Boucles

- **for: La fonction range()**

```
for iterator in range(nombre) :
    action(s)
```

- iterator prend par défaut les valeurs de 0 à **nombre – 1** (inclus)
- iterator s'incrémente de **1** à chaque tour de boucle.

```
for iterator in range(borne_inf, borne_sup) :
    action(s)
```

- iterator prend par défaut les valeurs de borne_inf à **borne_sup – 1** (inclus)
- iterator s'incrémente de **1** à chaque tour de boucle.

```
for iterator in range(borne_inf, borne_sup, pas) :
    action(s)
```

- iterator prend par défaut les valeurs de borne_inf à **borne_sup – 1** (inclus)
- iterator s'incrémente de **pas** à chaque tour de boucle.

Les structures de contrôle

Boucles

- **for: exemples**

range(nombre)	range(borne_inf, borne_sup)	range(borne_inf, borne_sup, pas)	str
`for i in range(4):` `  print(i)`	`for i in range(2, 7):` `  print(i)`	`for i in range(2, 10, 2):` `  print(i)`	`for i in "bonjour":` `  print(i)`
0 1 2 3	2 3 4 5 6	2 4 6 8	b o n j o u r